

## Capítulo 3

# Vacío de la intersección de una RCG con un lenguaje regular finito

El esquema por intersección del capítulo ?? reduce la satisfacibilidad de una fórmula a decidir si la intersección de dos lenguajes es vacía [4, 1].

El paso de las gramáticas libres del contexto y de las gramáticas de concatenación de rango simples a las RCG no conserva de manera automática la tratabilidad de ese enfoque. Bertsch y Nederhof prueban que, para RCG, el vacío de la intersección con un lenguaje regular arbitrario es indecidible. También prueban que, si el lenguaje regular es finito y se representa mediante un autómata acíclico, el problema de decidir si la intersección es no vacía es NP-completo [2]. En el presente capítulo se da un algoritmo de fuerza bruta para ese último caso.

Este algoritmo no sustituye el resultado de complejidad de Bertsch y Nederhof. Se precisa qué procedimiento de decisión se obtiene cuando el autómata es acíclico y se demuestra su correctitud. Su valor está en exponer qué ocurre al decidir el vacío con los dos formalismos por separado y en caracterizar su costo. Con él se decide el problema, pero no se obtiene una representación de la intersección que se pueda estudiar más allá de las propiedades del autómata. Por eso en el capítulo ?? se construye un objeto que, en un solo formalismo, expresa de manera simbólica<sup>1</sup> lo que reconocen ambos.

El capítulo se organiza de la siguiente manera. En la sección 3.1 se presentan los lenguajes regulares finitos. En la sección 3.2 se formalizan los autómatas acíclicos y sus caminos aceptantes. En la sección 3.3 se define el problema de vacío de la intersección. Por último, en la sección 3.4 se presenta el algoritmo de decisión, se demuestra su correctitud y se analiza su complejidad.

---

<sup>1</sup>Es decir, mediante la gramática misma, y no recorriendo los dos formalismos por separado.

### 3.1. Lenguajes regulares finitos

El enfoque SAT por intersección trabaja con un lenguaje regular que representa interpretaciones de una fórmula. Para una fórmula con una cantidad finita de ocurrencias de variables booleanas, ese lenguaje es finito. La presente sección fija la noción de lenguaje regular finito que será usada en el resto del capítulo.

**Definición 3.1.** Un lenguaje  $L$  es **finito** si existe un entero  $k \geq 0$  tal que  $L$  tiene exactamente  $k$  cadenas.

**Definición 3.2.** Un **lenguaje regular finito** es un lenguaje regular que también es finito.

Una familia de lenguajes finitos puede tener una representación mediante autómatas de tamaño polinomial y contener, al mismo tiempo, una cantidad exponencial de cadenas.

Por ejemplo, para cada  $n \geq 1$ , sea

$$L_n = \{0, 1\}^n.$$

El lenguaje  $L_n$  es finito y contiene  $2^n$  cadenas. No obstante, existe un autómata finito con  $n + 1$  estados que reconoce  $L_n$ : el autómata avanza de un estado al siguiente al leer 0 o 1, y acepta solo después de leer exactamente  $n$  símbolos.

El ejemplo anterior muestra por qué no basta con saber que el lenguaje regular es finito. La finitud garantiza que una enumeración termina, pero no garantiza que termine en tiempo polinomial respecto al tamaño de la representación. Cuando un lenguaje regular es finito, admite una representación mediante un autómata acíclico, que se define en la sección siguiente.

### 3.2. Autómatas acíclicos

En el presente capítulo se usa el autómata finito del capítulo ??, con una diferencia: la función de transición es parcial, no total, es decir,  $\delta(q, a)$  puede no estar definida para algunos pares  $(q, a)$ . Cuando  $\delta(q, a)$  no está definida, la lectura del símbolo  $a$  en el estado  $q$  no conduce a ningún estado.

**Definición 3.3.** Sean  $p_0, p_1, \dots, p_m$  estados de  $Q$  y  $a_1, a_2, \dots, a_m$  símbolos de  $\Sigma$ , con  $m \geq 0$ . Un **camino** desde  $p_0$  hasta  $p_m$  es una secuencia

$$\pi = p_0 a_1 p_1 a_2 p_2 \cdots a_m p_m$$

tal que, para cada  $1 \leq i \leq m$ , se cumple

$$\delta(p_{i-1}, a_i) = p_i.$$

La cadena que se lee a lo largo de  $\pi$ , denotada  $w(\pi)$ , es  $a_1 a_2 \cdots a_m$ . Si  $m = 0$ , el camino no contiene transiciones y la cadena que se lee es  $\varepsilon$ .

**Definición 3.4.** Un camino  $\pi$  en el autómata es **aceptante** si empieza en el estado inicial  $q_0$  y termina en un estado de aceptación. El lenguaje aceptado por  $A$  es

$$L(A) = \{w(\pi) \mid \pi \text{ es un camino aceptante de } A\}.$$

La aciclicidad se define a partir de los caminos del autómata.

**Definición 3.5.** Un autómata finito  $A$  es **acíclico** si ningún camino de  $A$  con al menos una transición empieza y termina en el mismo estado.

La aciclicidad es suficiente para asegurar que el lenguaje aceptado es finito. En los tres lemas siguientes se enuncian la cota de longitud de los caminos, que se usa en el análisis de complejidad, la finitud del lenguaje aceptado y la representación de todo lenguaje finito mediante un autómata acíclico.

**Lema 3.1.** Sea  $A = (Q, \Sigma, \delta, q_0, F)$  un autómata finito acíclico. Todo camino de  $A$  tiene longitud a lo sumo  $|Q| - 1$ . En consecuencia, toda cadena aceptada por  $A$  tiene longitud a lo sumo  $|Q| - 1$  [2].

**Lema 3.2.** Si  $A$  es un autómata finito acíclico, entonces  $L(A)$  es finito [2].

**Lema 3.3.** Todo lenguaje finito puede reconocerse mediante un autómata finito acíclico [2].

En el análisis de complejidad del presente capítulo, el autómata forma parte de la entrada, de modo que el costo se expresa en función de su tamaño. Ese costo depende, ante todo, de la cantidad de caminos aceptantes del autómata. Por ello se introduce la siguiente notación.

**Definición 3.6.** Sea  $A$  un autómata finito acíclico. Se denota por  $p(A)$  la cantidad de caminos aceptantes de  $A$ .

Como  $\delta$  asigna a lo sumo un estado a cada par  $(q, a)$ , cada cadena aceptada determina un único camino aceptante. En consecuencia,

$$p(A) = |L(A)|.$$

El autómata acíclico es la parte regular de la entrada; en la próxima sección se define el problema del vacío de la intersección.

### 3.3. Problema de vacío de la intersección

El **problema del vacío de la intersección de una RCG con un lenguaje regular finito** toma como entrada una RCG  $G$  y un autómata finito acíclico  $A$ , y pregunta si  $L(G) \cap L(A) \neq \emptyset$ .

La intersección es no vacía si y solo si existe una cadena que se reconoce a la vez por la RCG  $G$  y por el autómata  $A$ :

$$L(G) \cap L(A) \neq \emptyset \iff \exists w \in \Sigma^* \text{ tal que } w \in L(G) \text{ y } w \in L(A).$$

Decidir ese problema es el objetivo de la sección siguiente.

### 3.4. Algoritmo de decisión

En esta sección se presenta el algoritmo; su correctitud se demuestra en la subsección 3.4.1 y su complejidad se analiza en la subsección 3.4.2.

Como el autómata de entrada es acíclico, sus caminos aceptantes son finitos. Además, como  $\delta$  asigna a lo sumo un estado a cada par  $(q, a)$ , esos caminos están en correspondencia uno a uno con las cadenas aceptadas. Por tanto, una forma directa de decidir si la intersección es no vacía consiste en recorrer las cadenas aceptadas por el autómata y aplicar, a cada una, un algoritmo de reconocimiento para RCG. El procedimiento es determinista y, en general, no se ejecuta en tiempo polinomial.

Se usará como subrutina un procedimiento de reconocimiento para RCG, que se denota

$$\text{RCG-RECOGNIZE}(G, w).$$

La subrutina devuelve verdadero si y solo si  $w \in L(G)$ . El reconocimiento de una cadena por una RCG se resuelve en tiempo polinomial en la longitud de la cadena [3, 5].

Antes de enumerar cadenas conviene eliminar los estados que no pueden participar en ningún camino aceptante. Esa reducción evita recorrer ramas que no contribuyen al lenguaje del autómata.

**Definición 3.7.** Sea  $A = (Q, \Sigma, \delta, q_0, F)$  un autómata finito. Un estado  $q \in Q$  es **útil** si es alcanzable desde  $q_0$  y desde  $q$  se puede alcanzar algún estado de aceptación.

El conjunto de estados útiles se obtiene mediante dos recorridos del autómata. En el primero se determinan los estados alcanzables desde  $q_0$ : se parte de  $\{q_0\}$  y se agrega  $\delta(q, a)$  por cada estado  $q$  ya incluido y cada símbolo  $a$  para el que  $\delta(q, a)$  está definida, hasta que no se agregue ningún estado. En el segundo se determinan los estados desde los que se alcanza un estado de aceptación: se parte de  $F$  y se agrega cada estado  $q$  tal que  $\delta(q, a)$  ya está incluido para algún símbolo  $a$ , hasta que no se agregue ninguno. Un estado es útil cuando pertenece a los dos conjuntos. El autómata  $\text{USEFUL-PART}(A)$  tiene por estados los útiles, por estado inicial  $q_0$ , por estados de aceptación los de  $F$  que son útiles y por transiciones las  $\delta(q, a) = q'$  en las que  $q$  y  $q'$  son útiles. En cada recorrido, cada estado y cada transición se visitan a lo sumo una vez, de modo que la reducción se calcula en tiempo lineal en el tamaño del autómata.

El pseudocódigo de  $\text{USEFUL-PART}$  es el siguiente.

USEFUL-PART( $A$ )

```

1   $R = \{q_0\}$ 
2  while there is a transition  $\delta(q, a) = q'$  with  $q \in R$  and  $q' \notin R$ 
3       $R = R \cup \{q'\}$ 
4   $C = F$ 
5  while there is a transition  $\delta(q, a) = q'$  with  $q' \in C$  and  $q \notin C$ 
6       $C = C \cup \{q\}$ 
7   $U = R \cap C$ 
8  return  $(U, \Sigma, \delta', q_0, F \cap U)$ , where  $\delta'$  is the restriction of  $\delta$  to  $U$ 

```

El autómata  $\text{USEFUL-PART}(A)$  acepta el mismo lenguaje que  $A$ ; cuando no hay estados útiles, ese lenguaje es vacío.

El algoritmo principal se presenta a continuación.

NONEMPTY-INTERSECTION( $G, A$ )

```

1   $A' = \text{USEFUL-PART}(A)$ 
2  if the initial state of  $A'$  does not exist
3      return FALSE
4  return SEARCH( $q_0, \varepsilon$ )

```

La subrutina SEARCH recorre las cadenas aceptadas por el autómata. El parámetro  $q$  indica el estado actual y  $w$  almacena la cadena que se lee a lo largo del camino desde  $q_0$  hasta ese estado.

SEARCH( $q, w$ )

```

1  if  $q \in F$  and RCG-RECOGNIZE( $G, w$ )
2      return TRUE
3  for each symbol  $a \in \Sigma$  such that  $\delta'(q, a)$  is defined
4       $r = \delta'(q, a)$ 
5      if SEARCH( $r, wa$ )
6          return TRUE
7  return FALSE

```

En el pseudocódigo,  $\delta'$  denota la función de transición de  $A'$ . La línea 1 de SEARCH verifica la pertenencia a la RCG cada vez que el camino recorrido termina en un estado de aceptación. Si la verificación es positiva, la intersección es no vacía y el algoritmo termina. Si la verificación es negativa, el recorrido continúa, ya que desde un estado de aceptación pueden salir transiciones hacia otros caminos aceptantes. La línea 5 actualiza la cadena asociada al camino mediante la concatenación del símbolo leído.

La aciclicidad de  $A$  garantiza la terminación de la búsqueda. Los estados  $q_0, q_1, \dots, q_d$  que aparecen en una secuencia de llamadas recursivas anidadas forman un camino de  $A'$ : en cada paso, de la llamada en el estado  $q$  se pasa a la llamada en el estado  $\delta'(q, a)$ . La profundidad  $d$  de la recursión es entonces el número de transiciones de ese camino. Como  $A'$  es acíclico, ese camino no

contiene dos veces el mismo estado, de modo que ningún estado se repite a lo largo de la recursión. Por el lema 3.1,  $d \leq |Q| - 1$ . Como en cada llamada se generan a lo sumo  $|\Sigma|$  llamadas recursivas, el árbol de recursión es finito y la búsqueda termina.

A continuación se demuestra la correctitud del algoritmo.

### 3.4.1. Correctitud

La demostración separa tres hechos. El primero establece que toda cadena que se evalúa con RCG-RECOGNIZE pertenece al lenguaje del autómata. El segundo establece que toda cadena aceptada por el autómata se evalúa exactamente una vez. El tercero se apoya en la correctitud de RCG-RECOGNIZE.

**Lema 3.4.** *Toda cadena  $w$  para la que el algoritmo ejecuta RCG-RECOGNIZE( $G, w$ ) pertenece a  $L(A)$ .*

*Demostración.* La subrutina SEARCH ejecuta RCG-RECOGNIZE( $G, w$ ) solo en la línea 1, cuando el estado actual  $q$  pertenece a  $F$ . El valor de  $w$  se construye concatenando los símbolos que se leen desde  $q_0$  hasta  $q$ . Por tanto, existe un camino desde  $q_0$  hasta un estado de aceptación a lo largo del cual se lee  $w$ . Por la definición de aceptación,  $w \in L(A')$ . Como  $A' = \text{USEFUL-PART}(A)$  acepta el mismo lenguaje que  $A$ , se tiene  $L(A') = L(A)$ . Luego  $w \in L(A)$ .  $\square$

**Lema 3.5.** *Para toda cadena  $w \in L(A)$ , el algoritmo ejecuta RCG-RECOGNIZE( $G, w$ ) exactamente una vez.*

*Demostración.* Sea  $w = a_1a_2 \cdots a_m$  una cadena de  $L(A)$ . Como para cada par  $(q, a)$  hay a lo sumo un estado  $\delta(q, a)$ , existe un único camino de  $A$  que comienza en  $q_0$  y a lo largo del cual se lee  $w$ :

$$q_0a_1q_1a_2q_2 \cdots a_mq_m.$$

Además, como  $w \in L(A)$ , se tiene  $q_m \in F$ . Todos los estados de ese camino son útiles: cada uno es alcanzable desde  $q_0$  por el prefijo correspondiente, y desde cada uno se alcanza un estado de aceptación por el sufijo correspondiente. Por tanto, el camino también pertenece a  $A'$ .

La subrutina SEARCH recorre, para cada estado actual, todos los símbolos  $a$  para los cuales  $\delta'(q, a)$  está definida. Por inducción sobre la longitud de los prefijos de  $w$ , el algoritmo realiza una llamada recursiva asociada al único prefijo de longitud  $i$  de  $w$ , para cada  $0 \leq i \leq m$ . Al llegar al estado  $q_m$ , la cadena acumulada es  $w$  y se cumple  $q_m \in F$ . En ese momento se ejecuta RCG-RECOGNIZE( $G, w$ ).

La ejecución es única porque en  $A'$  hay a lo sumo un estado  $\delta'(q, a)$  por cada par  $(q, a)$ : ninguna cadena se lee a lo largo de dos caminos distintos desde  $q_0$ . Por tanto, el algoritmo no puede alcanzar un estado de aceptación con la misma cadena  $w$  por dos recorridos diferentes.  $\square$

**Lema 3.6.** *Para toda cadena  $w$ , la subrutina RCG-RECOGNIZE( $G, w$ ) devuelve verdadero si y solo si  $w \in L(G)$ .*

*Demostración.* El reconocimiento de una cadena por una RCG se decide en tiempo polinomial [3, 5]. RCG-RECOGNIZE es un procedimiento que resuelve ese problema y devuelve verdadero si y solo si  $w \in L(G)$ .  $\square$

**Teorema 3.1.** *El algoritmo NONEMPTY-INTERSECTION( $G, A$ ) devuelve verdadero si y solo si*

$$L(G) \cap L(A) \neq \emptyset.$$

*Demostración.* Supóngase primero que el algoritmo devuelve verdadero. Entonces, para alguna cadena  $w$ , se ejecutó RCG-RECOGNIZE( $G, w$ ) y la subrutina devolvió verdadero. Por el lema 3.4,  $w \in L(A)$ . Por el lema 3.6,  $w \in L(G)$ . Luego  $w \in L(G) \cap L(A)$ , y la intersección es no vacía.

Supóngase ahora que  $L(G) \cap L(A) \neq \emptyset$ . Entonces existe una cadena  $w$  tal que  $w \in L(G)$  y  $w \in L(A)$ . Por el lema 3.5, el algoritmo ejecuta RCG-RECOGNIZE( $G, w$ ) exactamente una vez. Por el lema 3.6, esa ejecución devuelve verdadero. Por tanto, el algoritmo devuelve verdadero.  $\square$

A continuación se analiza la complejidad del algoritmo.

### 3.4.2. Complejidad

El análisis depende de dos componentes. El primero es la enumeración de cadenas aceptadas por el autómata acíclico. El segundo es el reconocimiento de cada cadena mediante la RCG. En la presente subsección se expresan ambos costos de manera separada.

Sea  $|A| = |Q| + |\delta|$  el tamaño del autómata, donde  $|\delta|$  denota la cantidad de pares  $(q, a)$  para los cuales la transición está definida. Sea  $p(A)$  el número de caminos aceptantes de  $A$ . Como se estableció en la sección 3.2,  $p(A) = |L(A)|$ .

Después de eliminar estados no útiles, todo camino recorrido por el algoritmo es prefijo de algún camino aceptante. Como cada camino aceptante tiene longitud a lo sumo  $|Q| - 1$ , el número de prefijos recorridos está acotado por

$$p(A) |Q|.$$

Por tanto, el costo de la enumeración, sin contar las llamadas a RCG-RECOGNIZE, queda acotado por

$$O(p(A) |Q|).$$

Sea  $R_G(n)$  el tiempo que demora RCG-RECOGNIZE( $G, w$ ) sobre una cadena  $w$  de longitud  $n$ . Para una RCG fija,  $R_G(n)$  es polinomial en  $n$ . Si la gramática forma parte de la entrada, el costo es polinomial en la longitud de la cadena para cada gramática fijada, y el exponente depende de parámetros estructurales de la gramática, como la cantidad de variables que pueden recibir rangos en una cláusula [3, 5].

Por el lema 3.1, toda cadena aceptada tiene longitud a lo sumo  $|Q| - 1$ . Luego el costo total del algoritmo se acota por

$$O(|A| + p(A) |Q| + p(A) R_G(|Q| - 1)).$$

De forma equivalente,

$$O(|A| + p(A)(|Q| + R_G(|Q| - 1))).$$

La cota anterior es polinomial si  $p(A)$  está acotado por un polinomio en  $|A|$ . Sin embargo,  $p(A)$  no está polinomialmente acotado para todos los autómatas acíclicos. En la sección 3.1 ya se mostró un autómata así: el que reconoce  $L_n = \{0, 1\}^n$ , con  $n + 1$  estados, acíclico y con  $2^n$  cadenas aceptadas. Por la igualdad  $p(A) = |L(A)|$ , ese autómata tiene  $2^n$  caminos aceptantes con un tamaño lineal en  $n$ .

Por tanto, la enumeración determinista decide el problema, pero su costo puede ser exponencial en el tamaño del autómata, incluso para autómatas acíclicos.

El costo del algoritmo depende de cuántas cadenas acepta la representación regular. Si una clase de autómatas acíclicos tiene polinomialmente muchas cadenas aceptadas, el vacío de la intersección se decide en tiempo polinomial para cada RCG fija; si admite autómatas con exponencialmente muchas cadenas, ese costo es exponencial. Esa condición es trivial: enumerar es barato cuando hay poco que enumerar, de modo que la enumeración no es una vía factible para decidir el vacío de la intersección, pero sí necesaria para entender el problema y por qué conviene resolverlo sobre un único objeto que exprese lo que reconocen ambos formalismos.

Generar cada cadena aceptada en tiempo polinomial no es lo mismo que enumerarlas todas en tiempo polinomial. La cantidad de cadenas aceptadas puede ser exponencial, de modo que recorrerlas todas cuesta tiempo exponencial aun cuando cada una se obtenga en tiempo polinomial. Para decidir el vacío de la intersección hace falta, en el peor caso, recorrer todas: cuando la intersección es vacía, ninguna cadena aceptada pertenece a  $L(G)$ , y el recorrido solo concluye al agotarlas.

En este capítulo se definió el problema del vacío de la intersección de una RCG con un lenguaje regular finito y se dio un algoritmo de fuerza bruta que lo decide cuando el lenguaje regular se representa con un autómata acíclico; se demostró su correctitud y se caracterizó su costo, exponencial en el peor caso. En el próximo capítulo se modela el SAT como ese vacío, asociando a cada fórmula booleana un autómata booleano y una RCG de consistencia.



# Bibliografía

- [1] Manuel Aguilera López. Problema de la satisfacibilidad booleana de concatenación de rango simple. In *Jornada Científica Estudiantil MatCom 2016*, pages 1–5, La Habana, Cuba, 2016. Facultad de Matemática y Computación, Universidad de La Habana.
- [2] Eberhard Bertsch and Mark-Jan Nederhof. On the complexity of some extensions of RCG parsing. In *Proceedings of the Seventh International Workshop on Parsing Technologies (IWPT 2001)*, pages 66–77, Beijing, China, 2001.
- [3] Pierre Boullier. Proposal for a natural language processing syntactic backbone. Technical Report RR-3342, INRIA-Rocquencourt, France, 1998.
- [4] Alina Fernández Arias. El problema de la satisfacibilidad booleana libre del contexto. un algoritmo polinomial. Tesis de licenciatura, Facultad de Matemática y Computación, Universidad de La Habana, La Habana, Cuba, 2007.
- [5] Laura Kallmeyer. *Parsing Beyond Context-Free Grammars*. Cognitive Technologies. Springer, Berlin, Heidelberg, 2010.